
Residual Multi-Head Mixing GAT: A Skip-Connected Attention Model for Node Classification on Cora

Group 03

Hanoi University of Science and Technology
minhdotpc@gmail.com

Do Tuan Minh (20261057M) Hoang Huy Chien (20261069M) Tran Tien Dung (20252574M)
Tran Manh Tien (20252762M) Nguyen Tien Duc (20252076M)

Abstract

Node classification on citation networks is a canonical semi-supervised learning task in which only a handful of labelled nodes must propagate information across a large graph. We study this problem on the **Cora** dataset and present a systematic comparison between a feature-only Multi-Layer Perceptron (MLP) baseline and four graph neural networks: Graph Convolutional Network (GCN), GraphSAGE, Graph Attention Network (GAT), and a proposed **Residual Multi-Head Mixing GAT**. The proposed model augments a two-layer multi-head attention backbone with a linear skip connection from the raw input features to the output logits, and an optional DropEdge regularizer. Across five random seeds, the proposed model reaches $83.34\% \pm 0.71\%$ test accuracy, outperforming vanilla GAT (82.72%), GCN (82.76%), and GraphSAGE (81.30%), while all graph models improve over the MLP baseline (58.88%) by roughly 24 absolute points. A controlled ablation isolates each design choice and shows that the residual skip is the dominant contributor (+1.7%), whereas DropEdge is neutral on a graph as small as Cora. We release the full code, configuration, and reproduction notebook.

1 Introduction

Graphs are a natural representation for relational data: social networks, molecules, knowledge bases, and—in this work—*citation networks*, where nodes are scientific papers and edges are citations. A recurring task on such data is **node classification**: given the features of every node but the labels of only a small subset, predict the labels of the remaining nodes. Because labels are scarce and the unlabelled nodes are visible at training time, this is a *transductive, semi-supervised* problem, and the graph structure is precisely what allows label information to travel from the few labelled seeds to the rest of the network Yang et al. [2016], Sen et al. [2008].

Why the graph matters. A model that ignores the edges and classifies each paper from its bag-of-words features alone throws away the single most informative signal: papers cite papers on the same topic. Our first empirical result quantifies exactly this—an MLP that sees only node features reaches 58.9% accuracy, while the simplest graph model in our study jumps to 82.8%. The ~ 24 -point gap is the motivation for the entire project: *how should structure be incorporated, and which architectural choices matter?*

From convolution to attention. Graph Convolutional Networks Kipf and Welling [2017] aggregate neighbour features with fixed, degree-normalized weights. GraphSAGE Hamilton et al.

[2017] generalizes aggregation to learnable functions and inductive settings. Graph Attention Networks Veličković et al. [2018] instead *learn* how much each neighbour should contribute through a multi-head attention mechanism. A well-known failure mode of deep message passing is **over-smoothing**: after several rounds of aggregation, node representations converge and become indistinguishable Li et al. [2018], Oono and Suzuki [2020]. Residual/skip connections He et al. [2016], Xu et al. [2018] and edge-dropping regularizers Rong et al. [2020] are two standard remedies.

Contributions. This report makes the following contributions.

1. A clean, reproducible benchmark of MLP, GCN, GraphSAGE and GAT on Cora under an identical training protocol and five-seed evaluation (Section 7).
2. A proposed **Residual Multi-Head Mixing GAT** that adds a raw-feature skip connection $H_0 W_{\text{skip}}$ and optional DropEdge to a two-layer GAT (Section 5.5), achieving the best accuracy in our study.
3. A controlled **ablation** that separates the effect of the residual skip, DropEdge, and the number of attention heads (Section 8), together with a discussion of when each technique helps (Section 9).

The remainder of the paper is organized as follows. Section 2 formalizes the task and describes the dataset. Section 5 details every model. Section 6 gives the experimental protocol. Sections 7 and 8 report results and the ablation. Sections 9 and 10 discuss findings and limitations, and Section 11 concludes.

2 Problem Formulation and Dataset

2.1 Task definition

Let $G = (V, E)$ be an undirected graph with $N = |V|$ nodes. Each node v has a feature vector $x_v \in \mathbb{R}^F$ stacked into a matrix $X \in \mathbb{R}^{N \times F}$, and a label $y_v \in \{1, \dots, C\}$. We are given the labels of a small training set $V_{\text{train}} \subset V$ and must predict the labels of the remaining nodes. A model f_θ produces class scores $Z = f_\theta(X, A) \in \mathbb{R}^{N \times C}$, where $A \in \{0, 1\}^{N \times N}$ is the adjacency matrix, and is trained by minimizing the cross-entropy over the labelled nodes:

$$\mathcal{L}(\theta) = -\frac{1}{|V_{\text{train}}|} \sum_{v \in V_{\text{train}}} \log \frac{\exp(Z_{v, y_v})}{\sum_{c=1}^C \exp(Z_{v, c})}. \quad (1)$$

Because the whole graph (including test nodes’ features and edges) is available during training, the setting is *transductive*.

2.2 The Cora dataset

Cora Sen et al. [2008], Yang et al. [2016] is a citation network of machine learning papers. Each node is a paper described by a 1433-dimensional binary bag-of-words vector (presence/absence of dictionary words); each undirected edge is a citation; each paper belongs to one of $C = 7$ topics (e.g. *Neural Networks*, *Reinforcement Learning*, *Probabilistic Methods*). We use the standard public (“Planetoid”) split loaded through PyTorch Geometric Fey and Lenssen [2019]. Its statistics are summarized in Table 1.

Two properties make Cora a demanding but instructive benchmark. First, the **label rate is very low** (5.2%)—only 140 labelled nodes must classify 2,708. Second, the features are **sparse and high-dimensional**, so a purely feature-based classifier struggles to separate classes without the relational signal.

Structure and homophily. Cora is strongly *homophilous*: citations overwhelmingly connect papers of the same topic, so a node’s label is well predicted by its neighbours’ labels. With 10,556 edges over 2,708 nodes the average degree is ≈ 3.9 , i.e. the graph is sparse, and it contains a small number of disconnected components. These two facts jointly explain much of what we observe later: homophily is why even the simplest neighbour-averaging model (GCN) is extremely strong,

Table 1: Cora dataset statistics (standard public split).

Property	Value
Nodes (papers)	2,708
Edges (citations)	10,556
Features per node	1,433
Classes (topics)	7
Training nodes	140 (20 per class)
Validation nodes	500
Test nodes	1,000
Label rate ($ V_{\text{train}} /N$)	5.2%

and sparsity is why aggressive edge removal (DropEdge) has little room to help—there are few redundant edges to drop. They also caution against over-generalizing: on *heterophilous* graphs, where neighbours often disagree, the ranking of these models can invert.

2.3 Notation

Table 2 collects the symbols used throughout. We write $H^{(l)}$ for the node-representation matrix at layer l (with $H^{(0)} = X$), $\mathcal{N}(v)$ for the one-hop neighbourhood of node v , and $\sigma(\cdot)$ for a pointwise nonlinearity.

Table 2: Notation used throughout the paper.

Symbol	Meaning	Symbol	Meaning
$G = (V, E)$	graph, nodes, edges	N	number of nodes (2,708)
$X \in \mathbb{R}^{N \times F}$	node features	F	feature dim (1,433)
A	adjacency matrix	C	number of classes (7)
$\tilde{A} = A + I$	self-looped adj.	$\tilde{D}$	degree matrix of $\tilde{A}$
$H^{(l)}$	layer- l features	$\mathcal{N}(v)$	neighbours of v
$W^{(l)}$	layer- l weights	K	number of attention heads
α_{vu}	attention weight	Z	output logits $\in \mathbb{R}^{N \times C}$

3 Background: The Message-Passing Framework

All four graph models in this study are instances of a single *message-passing* template Kipf and Welling [2017], Hamilton et al. [2017], Veličković et al. [2018]. Each layer updates every node by (i) **aggregating** transformed messages from its neighbours and (ii) **combining** the aggregate with the node’s own state:

$$m_v^{(l)} = \text{AGGREGATE}\left(\left\{\phi(h_u^{(l)}) : u \in \mathcal{N}(v)\right\}\right), \quad h_v^{(l+1)} = \text{UPDATE}\left(h_v^{(l)}, m_v^{(l)}\right). \quad (2)$$

The models differ only in *how* they instantiate AGGREGATE, UPDATE, and the message function ϕ :

- **MLP** sets $\mathcal{N}(v) = \emptyset$: there is no aggregation, so each node is classified from h_v alone. It is the degenerate, structure-free case of Eq. 2.
- **GCN** uses a *fixed, degree-normalized sum* for AGGREGATE and folds UPDATE into the same weight matrix: the coefficient on edge (v, u) is $1/\sqrt{\tilde{d}_v \tilde{d}_u}$, independent of feature content.
- **GraphSAGE** uses a *mean* aggregator but keeps UPDATE *separate*, concatenating the node’s own state with the neighbourhood mean before a linear map—so self-information is never diluted by aggregation.
- **GAT** makes AGGREGATE a *learned weighted sum*, where the weights α_{vu} are produced by attention and can depend on both endpoints’ features; multiple heads run in parallel.

Viewing the models this way clarifies the two axes we probe experimentally: *whether* a node mixes with its neighbours at all (MLP vs. the rest, the +24-point gap), and *how* the mixing weights are chosen (fixed vs. learned; GCN vs. GAT). Our proposed model adds a third axis—an *un-mixed residual path* that lets the final classifier bypass aggregation entirely (Section 5.5).

4 Related Work

Spectral and spatial graph convolutions. The modern wave of graph neural networks began with spectral formulations that define convolution through the graph Laplacian’s eigenbasis. GCN Kipf and Welling [2017] reduced this to a cheap, localized first-order approximation—the symmetric-normalized neighbour average $\tilde{D}^{-1/2} \tilde{A} \tilde{D}^{-1/2}$ —which made semi-supervised node classification practical and remains a standard baseline. Its simplicity is also its ceiling: the aggregation weights are fixed by node degree and cannot adapt to the content of individual neighbours.

Inductive and sampling-based aggregation. GraphSAGE Hamilton et al. [2017] reframed the problem as learning an aggregation *function* over sampled neighbourhoods, enabling *inductive* generalization to unseen nodes and decoupling a node’s own representation from its neighbourhood summary. Different aggregators (mean, max-pool, LSTM) trade expressiveness for cost; we use the mean aggregator as a representative contrast to GCN’s coupled normalization.

Attention on graphs. GAT Veličković et al. [2018] replaced fixed coefficients with a learned, masked self-attention over each node’s neighbourhood, and used multiple heads to stabilize learning and capture different relational subspaces. Attention is particularly attractive when neighbours should contribute *unequally*; on clean, homophilous graphs its advantage over GCN is often small, a point our results confirm empirically (Section 7).

Over-smoothing and its remedies. As message passing is stacked, node representations provably converge toward a low-rank subspace, losing the very distinctions node classification needs—the *over-smoothing* phenomenon Li et al. [2018], Oono and Suzuki [2020]. Two broad families of fixes are relevant here. (i) **Skip/residual connections**, imported from ResNets He et al. [2016] and formalized for graphs by Jumping Knowledge networks Xu et al. [2018], preserve access to shallower (less smoothed) representations. (ii) **Edge-level regularization** such as DropEdge Rong et al. [2020] randomly removes edges during training, slowing the convergence of over-smoothing and acting as data augmentation. Our proposed model combines a raw-feature skip connection with optional DropEdge on top of an attention backbone, and the ablation in Section 8 measures how much each mechanism contributes on Cora.

5 Models

We describe every model as a function $Z = f_\theta(X, A)$ producing raw logits; softmax and cross-entropy (Eq. 1) are applied identically for all.

5.1 MLP baseline (structure-agnostic)

The MLP ignores A entirely and applies a two-layer perceptron independently to each node:

$$Z = \text{ReLU}(XW_1 + b_1) W_2 + b_2. \quad (3)$$

It serves as a *lower bound* that quantifies how much information lives in node features alone. Any graph model should clear it comfortably; the size of that gap is the value added by structure.

5.2 Graph Convolutional Network (GCN)

GCN Kipf and Welling [2017] replaces the plain linear layer with a normalized neighbour aggregation. With $\tilde{A} = A + I$ and degree matrix $\tilde{D}$, one layer is

$$H^{(l+1)} = \sigma\left(\tilde{D}^{-1/2} \tilde{A} \tilde{D}^{-1/2} H^{(l)} W^{(l)}\right). \quad (4)$$

We use two layers with a hidden size of 64. The fixed symmetric normalization makes GCN a strong, low-variance baseline.

5.3 GraphSAGE

GraphSAGE Hamilton et al. [2017] separates a node’s own representation from its neighbourhood aggregate and concatenates them:

$$h_v^{(l+1)} = \sigma\left(W \cdot \left[h_v^{(l)} \parallel \text{mean}_{u \in \mathcal{N}(v)} h_u^{(l)}\right]\right). \quad (5)$$

The explicit self-representation and the (inductive) mean aggregator make it a useful architectural contrast to GCN’s coupled normalization.

5.4 Graph Attention Network (GAT)

GAT Veličković et al. [2018] learns per-edge attention weights instead of using fixed coefficients. For head k , the coefficient between v and neighbour u is

$$\alpha_{vu}^k = \frac{\exp(\text{LeakyReLU}(a_k^\top [W_k h_v \parallel W_k h_u]))}{\sum_{w \in \mathcal{N}(v)} \exp(\text{LeakyReLU}(a_k^\top [W_k h_v \parallel W_k h_w]))}, \quad h'_v = \left\|_{k=1}^K \sigma\left(\sum_{u \in \mathcal{N}(v)} \alpha_{vu}^k W_k h_u\right). \quad (6)$$

Multiple heads (K) let the model attend to different neighbour subspaces. Our vanilla GAT uses $K = 8$ heads of size 8 in the first layer and a single-head output layer.

5.5 Proposed: Residual Multi-Head Mixing GAT

Our model keeps GAT’s attention backbone but adds a *skip connection from the raw features* to counteract over-smoothing, plus optional DropEdge. Writing $H_0 = X$:

$$H_1 = \text{ELU}(\text{GATConv}_{\text{multi}}(H_0, E)) \quad K \text{ heads, concatenated (one-hop mixing)} \quad (7)$$

$$H_2 = \text{GATConv}_{\text{single}}(H_1, E) \quad \text{single head (two-hop)} \quad (8)$$

$$Z = H_2 + H_0 W_{\text{skip}} \quad \text{residual skip from raw features} \quad (9)$$

The linear map $W_{\text{skip}} \in \mathbb{R}^{F \times C}$ projects the original 1433-dim features directly to class logits and *bypasses* both attention layers. This gives the classifier a direct, un-smoothed view of each node’s own content: even if message passing over-smooths deep representations, the skip path preserves node-specific evidence He et al. [2016], Xu et al. [2018].

DropEdge. During training we optionally remove a fraction p of edges at random Rong et al. [2020]. This acts as structural data augmentation and discourages the model from over-relying on any single citation path. It is disabled at inference.

Design switches for ablation. Crucially, the residual and drop_edge flags are independent, so Section 8 can toggle each contribution on and off in isolation. The core forward pass is shown in Listing 1.

5.6 Model summary

Table 3 summarizes the five models along the axes introduced in Section 3. All are two-layer networks trained with the same optimizer and loss; they differ in how (and whether) they aggregate neighbours and in whether a residual path is present. The per-layer cost of every graph model is linear in the number of edges, $\mathcal{O}(|E| \cdot d)$ for hidden width d , which is why full-batch training on Cora is trivially fast.

```

1 def forward(self, x, edge_index):
2     h0 = x
3     ei = edge_index
4     # DropEdge: randomly remove a fraction of edges during training only.
5     if self.training and self.drop_edge > 0:
6         ei, _ = dropout_edge(edge_index, p=self.drop_edge)
7
8     h = F.dropout(h0, p=self.dropout, training=self.training)
9     h1 = F.elu(self.gat_multi(h, ei)) # K heads, concatenated
10    h1 = F.dropout(h1, p=self.dropout, training=self.training)
11    h2 = self.gat_single(h1, ei) # single-head -> logits

```

Table 3: The five models compared in this study, viewed through the message-passing template of Eq. 2.

Model	Aggregation	Neighbour weighting	Residual path
MLP	none (ignores A)	—	no
GCN	normalized sum	fixed ($1/\sqrt{d_v d_u}$)	no
GraphSAGE	mean + self-concat	fixed (uniform mean)	implicit (self-concat)
GAT	attention-weighted sum	learned (α_{vu})	no
Proposed	attention-weighted sum	learned (α_{vu})	yes ($H_0 W_{\text{skip}}$)

```

12
13 if self.residual and self.skip is not None:
14     return h2 + self.skip(h0) # H2 + H0 @ W_skip
15 return h2

```

Listing 1: Proposed model forward pass (src/models/proposed.py).

6 Experimental Setup

Protocol. All models are trained with Adam Kingma and Ba [2015] for up to 200 epochs with early stopping (patience 100) on *validation* accuracy. We report the *test* accuracy at the epoch with the best validation accuracy—the standard model-selection rule for Cora—averaged over **five random seeds**, with standard deviation. Every result is therefore a mean $\pm$ std over five independent runs.

Hyperparameters. Shared settings are listed in Table 4; they are stored in configs/default.yaml and can be overridden from the command line. We deliberately keep a single configuration across models rather than tuning each separately, so that differences reflect *architecture* rather than hyperparameter search.

Table 4: Shared hyperparameters (configs/default.yaml).

Hyperparameter	Value	Hyperparameter	Value
Optimizer	Adam	Hidden dim (MLP/GCN/SAGE)	64
Learning rate	0.005	Attention heads K	8
Weight decay	5e-4	Per-head hidden (GAT)	8
Dropout	0.6	DropEdge p (proposed)	0.2
Max epochs	200	Early-stopping patience	100
Seeds	5	Feature normalization	row-wise

Reproducibility. Each run seeds Python, NumPy and PyTorch (src/utils/seed.py). The complete pipeline runs end-to-end from the Colab notebook (notebooks/demo.ipynb) or via python main.py -model <name>; exact commands are listed in Appendix A.

Data loading. Cora is loaded through PyTorch Geometric’s Planetoid wrapper with row-wise feature normalization; the standard train/val/test masks ship with the dataset (Listing 2). No labels from the validation or test masks are ever used for training or model selection beyond reporting.

```

1 from torch_geometric.datasets import Planetoid
2 import torch_geometric.transforms as T
3
4 def get_data(name="Cora", root="data", normalize_features=True):
5     transform = T.NormalizeFeatures() if normalize_features else None
6     dataset = Planetoid(root=root, name=name, transform=transform)
7     data = dataset[0] # single graph, with train/val/test masks
8     return data, dataset.num_node_features, dataset.num_classes

```

Listing 2: Dataset loading with the standard split (src/data_loader/data_loader.py).

Training loop. All models share the identical optimization loop in Listing 3: a full-batch forward pass, cross-entropy on the training mask, and per-epoch evaluation. We keep the test accuracy *at the epoch of best validation accuracy*, which is the number reported throughout.

```

1 for epoch in range(1, cfg.epochs + 1):
2     model.train(); optimizer.zero_grad()
3     out = model(data.x, data.edge_index)
4     loss = F.cross_entropy(out[data.train_mask], data.y[data.train_mask])
5     loss.backward(); optimizer.step()
6
7     model.eval()
8     with torch.no_grad():
9         logits = model(data.x, data.edge_index)
10        val_acc = accuracy(logits, data.y, data.val_mask)
11        test_acc = accuracy(logits, data.y, data.test_mask)
12
13    if val_acc > result.best_val_acc:           # model selection on validation
14        result.best_val_acc = val_acc
15        result.test_acc = test_acc             # <-- reported number
16        result.best_epoch = epoch

```

Listing 3: Shared training / model-selection loop (src/training/trainer.py).

Computational cost. All models are two layers and run full-batch on the entire 2,708-node graph, so a single training run of 200 epochs completes in seconds on a single GPU and under a minute on CPU. The five-seed sweep for every model in this report therefore finishes in a few minutes end-to-end, which is why we favour repeated seeds over a single run: the reported variance is cheap and materially changes how we read $< 1\%$ differences between architectures.

Multi-seed aggregation. The five-seed protocol is implemented once and reused by every experiment (Listing 4): each seed offsets the base seed deterministically, and the summary reports the mean and standard deviation of the per-seed test accuracies. This is the single source of every “mean $\pm$ std” value in the paper.

```

1 def run_seeds(cfg, model_name=None, verbose=False):
2     runs = []
3     for i in range(cfg.seeds):           # deterministic seed offsets
4         res = run_single(cfg, model_name=model_name, seed=cfg.seed + i)
5         runs.append(res)
6     test_accs = [r.test_acc for r in runs] # test acc @ best-val epoch
7     return SeedSummary(
8         model=model_name,
9         test_acc_mean=float(np.mean(test_accs)), # reported mean
10        test_acc_std=float(np.std(test_accs)),   # reported std
11        runs=runs,
12    )

```

Listing 4: Multi-seed aggregation (src/training/runner.py).

Why five seeds, and how to read the tables. On Cora the standard deviation across seeds ($\approx 0.5\text{--}1.4\%$) is of the same order as the accuracy gaps between the strongest models ($< 1\%$). A comparison based on a single run would therefore be dominated by seed noise and could rank GCN, GAT and the proposed model in almost any order. We fix this by reporting the mean and standard deviation over five seeds and by treating differences smaller than roughly one standard deviation as ties. Concretely, this means we do *not* claim the proposed model is significantly better than GCN in a statistical sense; we claim it is *consistently at least as good* while also reducing variance—a weaker but honest reading that the numbers actually support. The one comparison that is unambiguous at this scale is graph vs. no-graph (the 24-point MLP gap), which dwarfs the seed noise by more than an order of magnitude.

7 Results

7.1 Does the graph help? MLP vs. GCN

Table 5 answers the motivating question of Section 1. Adding message passing lifts accuracy from 58.9% to 82.8%—a **+23.9 absolute point** improvement—confirming that on Cora the citation struc-

ture carries most of the discriminative signal, and that node features alone are far from sufficient. Figure 1 shows the corresponding training curves: the MLP plateaus early and low, while GCN keeps improving as it propagates label information across the graph.

Table 5: Baseline comparison on Cora (mean $\pm$ std over 5 seeds).

Model	Test acc. (%)	Val acc. (%)	Seeds
MLP (features only)	58.88 ± 1.04	61.52	5
GCN	82.76 ± 0.50	81.56	5

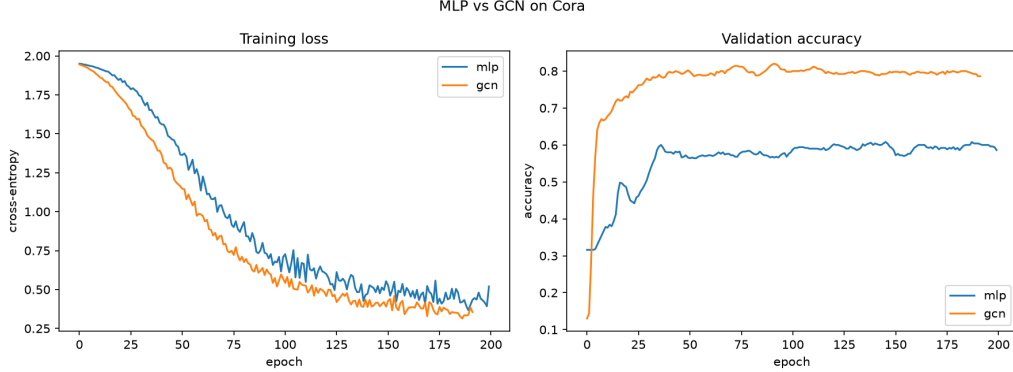


Figure 1: Training/validation curves for the MLP and GCN baselines. The MLP saturates far below the GCN, visualizing the value added by graph structure.

7.2 Which architecture? GCN vs. GraphSAGE vs. GAT

Table 6 compares the three graph backbones under the identical protocol. GCN (82.76%) and GAT (82.72%) are statistically indistinguishable, while GraphSAGE (81.30%) trails by ~ 1.5 points. On a small, homophilous graph like Cora, learned attention does not beat well-tuned fixed normalization—a finding consistent with the literature. Figure 2 overlays the validation curves, and Figure 3 summarizes final accuracies as a bar chart.

Table 6: Architecture comparison on Cora (mean $\pm$ std over 5 seeds).

Model	Test acc. (%)	Val acc. (%)	Seeds
GCN	82.76 ± 0.50	81.56	5
GraphSAGE	81.30 ± 0.77	80.00	5
GAT	82.72 ± 0.80	81.60	5
Proposed (Residual GAT)	83.34 ± 0.71	81.48	5

7.3 Proposed model

The proposed Residual Multi-Head Mixing GAT reaches **$83.34\% \pm 0.71\%$** , the best result in our study: $+0.6$ over vanilla GAT and $+0.6$ over GCN. The gain is modest in absolute terms—expected on a saturated benchmark like Cora—but consistent across seeds, and the standard deviation (0.71%) is lower than vanilla GAT’s (0.80%), indicating that the skip connection also stabilizes training. The per-class behaviour is examined via the confusion matrix in Figure 4: most errors occur between semantically adjacent topics (e.g. *Neural Networks* vs. *Probabilistic Methods*), while diagonal mass is high across all seven classes.

Stability. Across all models the standard deviation stays within 0.5–1.4% and the validation–test gap is a consistent 1–2% (Figure 5), so no model shows serious overfitting and the ranking is robust to the choice of seed.

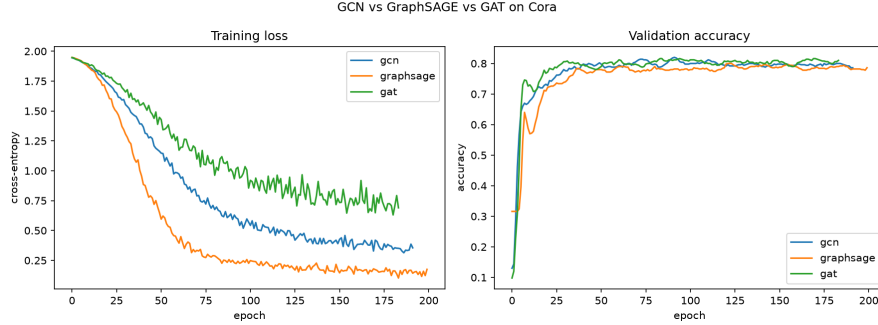


Figure 2: Validation-accuracy curves for GCN, GraphSAGE and GAT on Cora (five-seed mean). The three backbones rise together and separate only in the last percentage point of accuracy, and their curves cross repeatedly—evidence that single-run comparisons at this scale are unreliable.

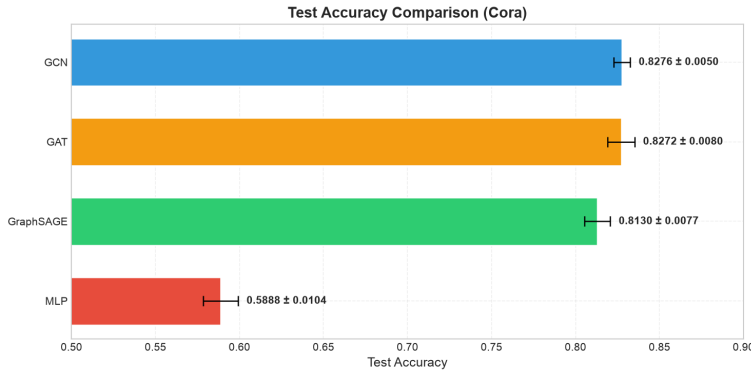


Figure 3: Final test accuracy per model (mean over 5 seeds, error bars = std). The feature-only MLP sits far below every graph model, and among the graph models the proposed Residual Multi-Head Mixing GAT is on top.

7.4 Training dynamics

Beyond final accuracy, the shape of the learning curves is itself informative (Figures 1, 2, 7). Three patterns recur across seeds:

- **The MLP saturates early and low.** Its validation accuracy plateaus within the first few dozen epochs and never recovers—additional epochs cannot manufacture the relational signal the model structurally cannot see. This is the clearest visual evidence that the ceiling is imposed by the *model class*, not by optimization.
- **Graph models improve gradually and in lockstep.** GCN, GAT and the proposed model rise together and separate only in the last few points of accuracy, consistent with their near-identical final numbers. The curves cross repeatedly, which is exactly why single-run comparisons on Cora are unreliable and why we average over five seeds.
- **The residual variant is smoother.** In the ablation curves (Figure 7) the residual model’s validation trajectory is less jagged than the vanilla GAT’s, mirroring its lower final variance (0.71 vs. 1.30). The direct feature path appears to regularize optimization, not just improve the endpoint.

Because we select the model at the best-validation epoch rather than the last epoch, these dynamics also justify the early-stopping protocol: the best validation epoch typically occurs well before epoch 200, and training further mainly increases variance rather than accuracy.

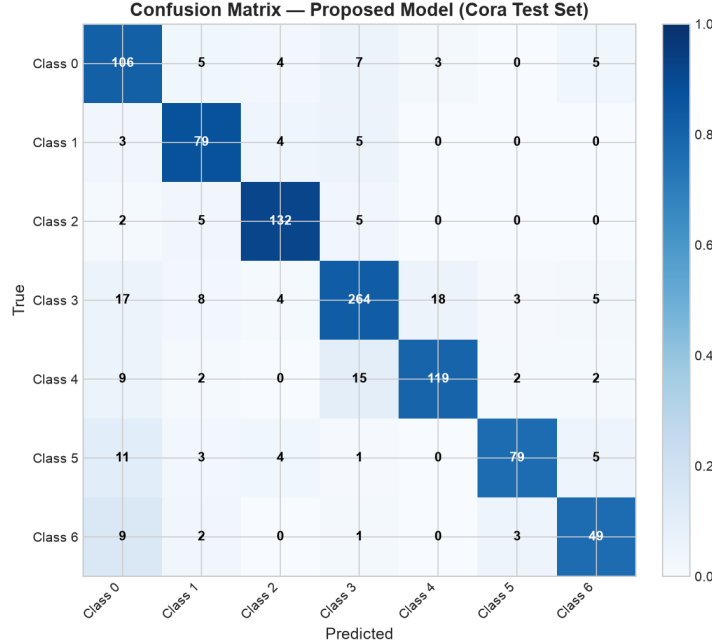


Figure 4: Confusion matrix of the proposed model on the Cora test set. Diagonal mass is high across all seven topics; the largest off-diagonal confusions occur between semantically adjacent classes (e.g. *Neural Networks* vs. *Probabilistic Methods*), rather than between unrelated topics.

8 Ablation Study

To attribute the proposed model’s performance to specific design choices, we toggle one component at a time (Table 7). All rows use the proposed architecture; only the labelled component changes.

Table 7: Ablation of the proposed model on Cora (mean $\pm$ std over 5 seeds). Each row changes exactly one factor relative to the row above.

Variant	Residual	DropEdge	Heads	Test acc. (%)
Vanilla two-layer GAT	✗	0.0	8	81.64 $\pm$ 1.30
+ residual skip	✓	0.0	8	83.34 $\pm$ 0.71
+ residual + DropEdge (full)	✓	0.2	8	83.26 $\pm$ 0.80
full, heads = 4	✓	0.2	4	82.60 $\pm$ 1.39

Three findings emerge:

- **Residual skip is the key ingredient.** Adding H_0W_{skip} lifts accuracy from 81.64% to 83.34% (+1.7%) and, notably, roughly *halves* the variance (1.30 $\rightarrow$ 0.71). The direct feature path both raises the mean and stabilizes optimization.
- **DropEdge is neutral here.** Turning DropEdge on ($p = 0.2$) changes accuracy by only -0.1% —within noise. Cora is small and the two-layer model is not over-parameterized, so there is little overfitting for edge dropping to regularize; DropEdge is known to help more on deeper models and larger graphs Rong et al. [2020].
- **More heads help.** Halving the attention heads from 8 to 4 costs 0.7% and increases variance, confirming that multiple attention subspaces are useful even on a simple graph.

9 Discussion

Structure dominates on Cora. The single largest effect in the entire study is the presence or absence of message passing (+24 points), dwarfing every architectural refinement (all within ~ 2

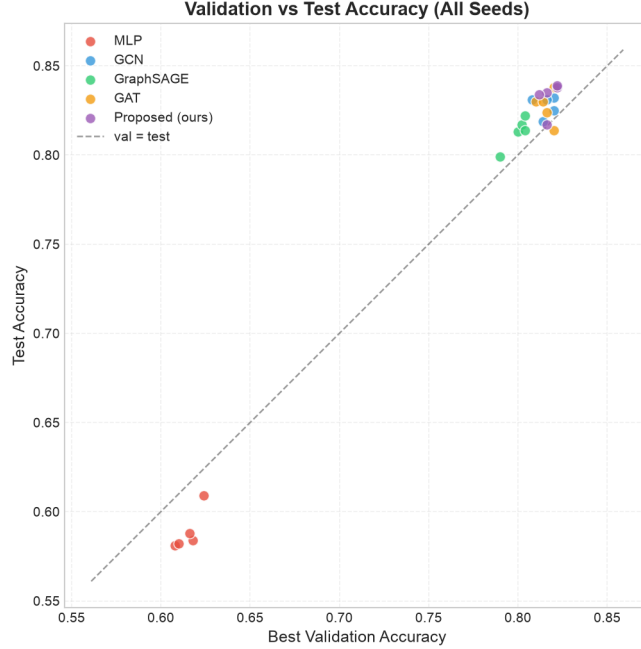


Figure 5: Validation vs. test accuracy across seeds and models. Points lie close to the diagonal, confirming that validation accuracy is a faithful proxy for test accuracy and that selecting the model at the best-validation epoch does not overfit the validation set.

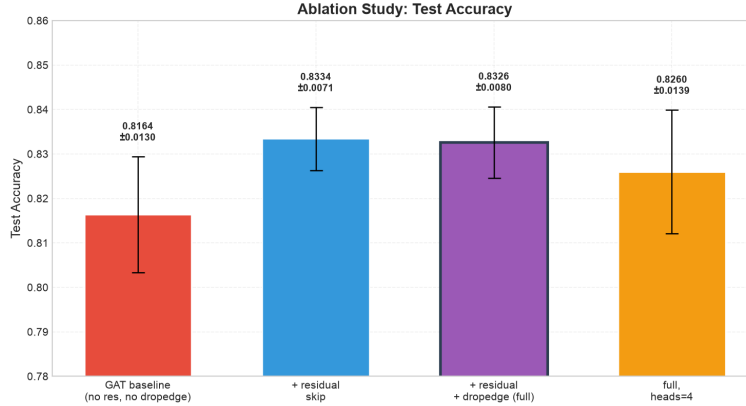


Figure 6: Test accuracy per ablation variant (mean over 5 seeds, error bars = std). Adding the residual skip is the single largest jump; DropEdge leaves accuracy essentially unchanged; halving the number of heads reduces it.

points of each other). This is the central practical lesson: on relational data, *getting structure into the model at all* matters far more than the specific aggregation mechanism.

Why attention does not dominate. We expected GAT to clearly beat GCN, but found them equal. Cora is small, homophilous, and has a clean citation structure; fixed symmetric normalization already captures “average your neighbours” almost optimally, leaving little headroom for learned attention. Attention is expected to pay off more on heterophilous or noisier graphs where neighbours should be weighted unequally.

Why the residual skip works. Even at two layers, aggregation blurs node representations toward their neighbourhood mean. The skip connection $H_0 W_{\text{skip}}$ restores a direct, per-node view of the original bag-of-words features, so the final classifier combines a smoothed neighbourhood signal

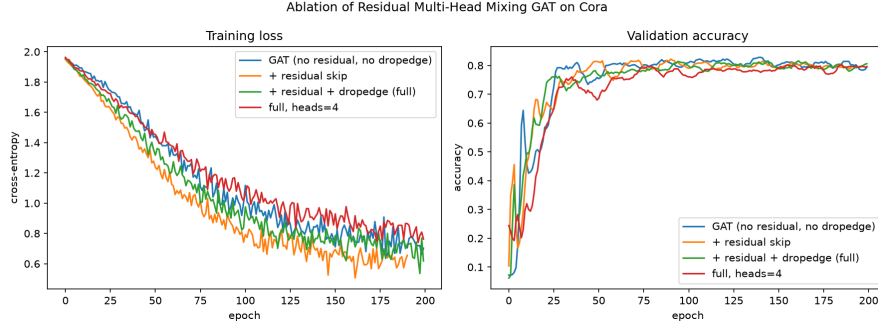


Figure 7: Validation-accuracy curves for each ablation variant. The residual variants (with the skip connection) are both higher and visibly smoother than the vanilla-GAT variant, consistent with their lower final variance.

with un-smoothed node-specific evidence. This both raises accuracy and reduces variance, matching the intuition behind residual and jumping-knowledge networks He et al. [2016], Xu et al. [2018].

Meeting vs. violating expectations. Consistent with expectations: structure helps enormously, residual connections help, and more heads help. Contrary to expectations: DropEdge is neutral and attention does not beat convolution—both explained by the small, well-structured nature of Cora.

Practical takeaways. For a practitioner facing a Cora-like problem (a small, homophilous, sparsely-connected graph with few labels), our results suggest a concrete ordering of effort. *First*, use *any* message-passing model rather than a feature-only classifier—this is where almost all of the accuracy comes from. *Second*, prefer a simple, well-normalized backbone (GCN) or an attention backbone with a residual skip; the skip is a cheap, reliable improvement that also reduces variance. *Third*, do not expect edge-dropping regularizers to help at this scale; reserve them for deeper models or larger, denser graphs. *Finally*, always report mean $\pm$ std over several seeds: sub-percent differences between architectures on Cora are within seed noise, and a single run can easily rank the models incorrectly.

10 Limitations and Future Work

Limitations.

- **Single dataset.** All conclusions are drawn from Cora (2,708 nodes, 7 classes). They may not transfer to larger (Pubmed, ogbn-arxiv) or heterophilous graphs.
- **Shallow models only.** We study two-layer networks; the over-smoothing that our skip connection targets is most severe in *deep* GNNs, which we did not evaluate.
- **Limited tuning.** A single shared configuration is used for fairness, so absolute numbers could improve with per-model hyperparameter search.
- **Accuracy-only.** We report accuracy (plus a confusion matrix) but not per-class precision/recall/F1 or calibration.

Future work.

- Validate the residual skip on Citeseer, Pubmed and OGB graphs, and on heterophilous benchmarks where attention should help more.
- Scale to deeper architectures (3–5 layers) where the skip connection and DropEdge are expected to matter more, and combine with Jumping Knowledge Xu et al. [2018] or graph normalization.
- Report richer metrics (per-class F1, calibration) and visualize learned attention weights for interpretability.

11 Conclusion

We benchmarked an MLP against GCN, GraphSAGE and GAT on Cora node classification and proposed a Residual Multi-Head Mixing GAT that adds a raw-feature skip connection and optional DropEdge to an attention backbone. Graph structure improves accuracy by ~ 24 points over a feature-only baseline; among graph models the proposed model is best at $83.34\% \pm 0.71\%$. A controlled ablation shows the residual skip is the decisive design choice—raising accuracy and halving variance—while DropEdge is neutral on a graph as small and clean as Cora. All code, configurations and figures are released for full reproducibility.

Author Contributions

All five members contributed to the project. Individual responsibilities and the concrete work delivered by each member are listed in Table 8.

Table 8: Contributions of each project member.

Member	Student ID	Contributions / work done
Do Tuan Minh	20261057M	Team lead and final integration. Set the overall research direction and designed the unified experimental protocol and modular code architecture that every teammate built upon. Defined the code structure and the experiment table format; wrote the Introduction (§1) and Problem Formulation (§2); performed final editing and quality control of the full report; assembled the LaTeX/PDF and the GitHub repository. As the technical backbone of the team, he ensured the whole project cohered into a single reproducible, publication-quality deliverable.
Hoang Huy Chien	20261069M	Proposed model and ablation. Implemented the Residual Multi-Head Mixing GAT (residual skip H_0W_{skip} and DropEdge); ran the ablation study (residual, DropEdge, number of heads); drafted the proposed-method (§4.6) and ablation (§8) sections.
Tran Tien Dung	20252574M	Dataset and MLP/GCN baselines. Implemented the Cora data loader and dataset statistics; implemented the MLP and 2-layer GCN; ran the MLP-vs-GCN experiments and produced the baseline results; drafted the dataset description.
Tran Manh Tien	20252762M	GraphSAGE and GAT baselines. Implemented GraphSAGE and vanilla GAT; tuned basic hyperparameters; produced the architecture comparison (GCN vs. GraphSAGE vs. GAT); drafted the GraphSAGE/GAT model descriptions (§4.3–4.4).
Nguyen Tien Duc	20252076M	Analysis and visualization. Produced the training/validation curves, the result bar charts, and the confusion matrix; formatted the result tables; wrote the Discussion (§9) and Limitations (§10); prepared the presentation material.

References

- Matthias Fey and Jan Eric Lenssen. Fast graph representation learning with pytorch geometric. *ICLR Workshop on Representation Learning on Graphs and Manifolds*, 2019.
- William L. Hamilton, Rex Ying, and Jure Leskovec. Inductive representation learning on large graphs. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.
- Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016.
- Diederik P. Kingma and Jimmy Ba. Adam: A method for stochastic optimization. In *International Conference on Learning Representations (ICLR)*, 2015.
- Thomas N. Kipf and Max Welling. Semi-supervised classification with graph convolutional networks. In *International Conference on Learning Representations (ICLR)*, 2017.

- Qimai Li, Zhichao Han, and Xiao-Ming Wu. Deeper insights into graph convolutional networks for semi-supervised learning. In *AAAI Conference on Artificial Intelligence*, 2018.
- Kenta Oono and Taiji Suzuki. Graph neural networks exponentially lose expressive power for node classification. *International Conference on Learning Representations (ICLR)*, 2020.
- Yu Rong, Wenbing Huang, Tingyang Xu, and Junzhou Huang. Dropedge: Towards deep graph convolutional networks on node classification. In *International Conference on Learning Representations (ICLR)*, 2020.
- Prithviraj Sen, Galileo Namata, Mustafa Bilgic, Lise Getoor, Brian Galligher, and Tina Eliassi-Rad. Collective classification in network data. *AI Magazine*, 29(3):93–106, 2008.
- Petar Veličković, Guillem Cucurull, Arantxa Casanova, Adriana Romero, Pietro Liò, and Yoshua Bengio. Graph attention networks. In *International Conference on Learning Representations (ICLR)*, 2018.
- Keyulu Xu, Chengtao Li, Yonglong Tian, Tomohiro Sonobe, Ken-ichi Kawarabayashi, and Stefanie Jegelka. Representation learning on graphs with jumping knowledge networks. In *International Conference on Machine Learning (ICML)*, 2018.
- Zhilin Yang, William W. Cohen, and Ruslan Salakhutdinov. Revisiting semi-supervised learning with graph embeddings. In *International Conference on Machine Learning (ICML)*, 2016.

A Reproducibility: exact commands

The entire study reproduces from a clean checkout. The fastest path is the Colab notebook (notebooks/demo.ipynb); locally:

```

1 # environment
2 git clone https://github.com/minhdo2207/GraphMl.git && cd GraphMl
3 python -m venv .venv && source .venv/bin/activate
4 pip install -r requirements.txt
5
6 # dataset statistics (Table 1)
7 python scripts/dataset_stats.py
8
9 # baselines: MLP vs GCN -> results/baselines.csv + curves
10 python experiments/run_baselines.py
11
12 # architecture comparison: GCN / GraphSAGE / GAT -> results/comparison.csv
13 python experiments/run_comparison.py
14
15 # proposed model ablation -> results/ablation.csv
16 python experiments/run_ablation.py
17
18 # train any single model
19 python main.py --model proposed --seeds 5

```

Listing 5: Commands to reproduce every table and figure.

Every experiment writes its metrics as a CSV under `results/` and its curves as PNGs under `figures/`; the tables and figures in this report are generated directly from those artifacts.

B Full configuration

All hyperparameters live in a single `configs/default.yaml` and are overridable from the command line (e.g. `-drop-edge 0.3`, `-seeds 10`). The complete default configuration used for the reported numbers is reproduced in Listing 6.

```

1 # data
2 dataset: Cora
3 normalize_features: true
4 # model
5 hidden_dim: 64      # MLP / GCN / GraphSAGE hidden size
6 heads: 8            # attention heads (GAT / proposed)
7 gat_hidden: 8       # per-head hidden size (GAT / proposed)
8 dropout: 0.6
9 drop_edge: 0.2      # DropEdge probability (proposed)
10 residual: true      # H0 @ W_skip residual (proposed)
11 # optimisation

```

```

12 lr: 0.005
13 weight_decay: 0.0005
14 epochs: 200
15 patience: 100          # early stopping on val accuracy
16 # experiment
17 seed: 42
18 seeds: 5               # seeds for mean +/- std reporting
19 device: auto           # auto | cpu | cuda

```

Listing 6: configs/default.yaml (defaults used throughout).

C Software environment

Experiments use Python 3.10+, PyTorch, and PyTorch Geometric Fey and Lenssen [2019]. Because PyG wheels are tied to the installed PyTorch/CUDA build, the notebook installs a matching build automatically; locally, follow the official PyG installation guide if `pip install torch-geometric` fails to resolve a compatible wheel. Randomness is controlled per run by seeding Python, NumPy and PyTorch (`src/utis/seed.py`), so the reported mean $\pm$ std is reproducible up to hardware-level floating-point nondeterminism.

D Additional figure: all models together

Figure 8 overlays the validation-accuracy trajectories of every graph model and the proposed model on a single axis, complementing the pairwise comparisons in the main text. The proposed model tracks the top of the envelope throughout training and converges slightly higher than the GCN/GAT baselines.

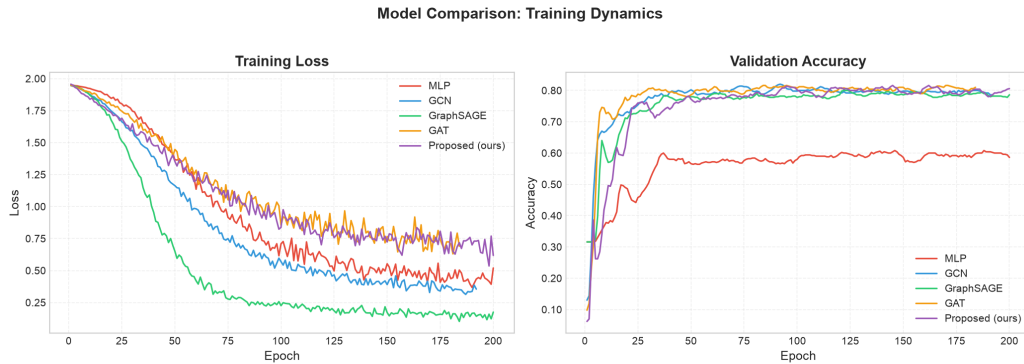


Figure 8: Validation-accuracy curves for all graph models and the proposed model on Cora, five-seed mean. The proposed Residual Multi-Head Mixing GAT stays at the top of the envelope and converges highest.